

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 6
По дисциплине «Современные платформы программирования»

Выполнил:
Шевчук А. В.
Студент группы ПО-13
Проверил:
Кулик А. Д.

Цель работы: освоить приемы тестирования кода на примере использования пакета `pytest`

Ход работы:

Задание 1: Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`
`@pytest.fixture`

```
def empty_cart():  
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item):
```

```
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
```

```

coupons = {"SAVE10": 10, "HALF": 50}

if coupon_code in coupons:
    cart.apply_discount(coupons[coupon_code])
else:
    raise ValueError("Invalid coupon")

```

- Напишите тесты на `apply_coupon`
- Замокните словарь `coupons` с помощью `monkeypatch` или `patch.dict`

shopping.py:

```

class Requests:
    """Заглушка для имитации requests.post."""

    @staticmethod
    def post(url, json=None):
        """Имитирует отправку POST-запроса."""
        return {"url": url, "json": json}

requests = Requests()

class Cart:
    """Корзина покупок."""

    def __init__(self):
        """Создает пустую корзину."""
        self.items = []

    def add_item(self, name, price):
        """Добавляет товар в корзину."""
        if price < 0:
            raise ValueError("Price cannot be negative")

        self.items.append({"name": name, "price": price})

    def total(self):
        """Возвращает общую стоимость товаров."""
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount):
        """Применяет скидку к товарам."""
        if discount < 0 or discount > 100:
            raise ValueError("Invalid discount")

        for item in self.items:
            item["price"] = item["price"] * (1 - discount / 100)

COUPONS = {"SAVE10": 10, "HALF": 50}

def log_purchase(item):
    """Логирует покупку в удаленную систему."""
    requests.post("https://example.com/log", json=item)

def apply_coupon(cart, coupon_code):
    """Применяет купон к корзине."""
    if coupon_code not in COUPONS:
        raise ValueError("Invalid coupon")

    cart.apply_discount(COUPONS[coupon_code])

```

test_cart.py:

```

# pylint: disable=redefined-outer-name,import-error

"""Тесты для мини-библиотеки покупок."""

```

```

from unittest.mock import patch

import pytest

import shopping
from shopping import Cart, apply_coupon, log_purchase

@pytest.fixture
def empty_cart():
    """Создает пустую корзину."""
    return Cart()

def test_add_item(empty_cart):
    """Проверяет добавление товара."""
    empty_cart.add_item("Apple", 10.0)

    assert len(empty_cart.items) == 1

def test_add_item_negative_price(empty_cart):
    """Проверяет ошибку при отрицательной цене."""
    with pytest.raises(ValueError):
        empty_cart.add_item("Apple", -10.0)

def test_total(empty_cart):
    """Проверяет вычисление общей стоимости."""
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Milk", 20.0)

    assert empty_cart.total() == 30.0

@pytest.mark.parametrize(
    ("discount", "expected"),
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
    ],
)
def test_apply_discount(empty_cart, discount, expected):
    """Проверяет применение скидки."""
    empty_cart.add_item("Apple", 100.0)

    empty_cart.apply_discount(discount)

    assert empty_cart.total() == expected

@pytest.mark.parametrize("discount", [-1, 101])
def test_apply_discount_invalid(empty_cart, discount):
    """Проверяет ошибку при неверной скидке."""
    empty_cart.add_item("Apple", 100.0)

    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)

@patch("shopping.requests.post")
def test_log_purchase(mock_post):
    """Проверяет логирование покупки без реального HTTP-запроса."""
    item = {"name": "Apple", "price": 10.0}

    log_purchase(item)

    mock_post.assert_called_once_with("https://example.com/log", json=item)

def test_apply_coupon_save10(empty_cart):
    """Проверяет купон SAVE10."""
    empty_cart.add_item("Apple", 100.0)

    apply_coupon(empty_cart, "SAVE10")

    assert empty_cart.total() == 90.0

def test_apply_coupon_half(empty_cart):
    """Проверяет купон HALF."""
    empty_cart.add_item("Apple", 100.0)

    apply_coupon(empty_cart, "HALF")

    assert empty_cart.total() == 50.0

def test_apply_coupon_invalid(empty_cart):
    """Проверяет ошибку при неверном купоне."""
    empty_cart.add_item("Apple", 100.0)

    with pytest.raises(ValueError):

```

```

        apply_coupon(empty_cart, "BAD")

def test_apply_coupon_monkeypatch(empty_cart, monkeypatch):
    """Проверяет подмену словаря купонов."""
    monkeypatch.setattr(shopping, "COUPONS", {"TEST": 25})
    empty_cart.add_item("Apple", 100.0)

    apply_coupon(empty_cart, "TEST")

    assert empty_cart.total() == 75.0

```

```

C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6>python -m pytest src/proj1
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6
plugins: anyio-4.13.0
collected 13 items

src\proj1\test_cart.py ..... [100%]

===== 13 passed in 0.07s =====

```

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

lab1_functions.py:

```

"""Функции из лабораторной работы 1."""

def parse_numbers(text):
    """Преобразует строку в список целых чисел."""
    if not text.split():
        raise ValueError("Empty input")

    return [int(number) for number in text.split()]

def calculate_median(numbers):
    """Вычисляет медиану списка чисел."""
    if not numbers:
        raise ValueError("Empty list")

    sorted_numbers = sorted(numbers)
    middle = len(sorted_numbers) // 2

    if len(sorted_numbers) % 2 == 1:
        return sorted_numbers[middle]

    return (sorted_numbers[middle - 1] + sorted_numbers[middle]) / 2

def parse_digits(text):
    """Преобразует строку в список цифр."""
    if not text.split():
        raise ValueError("Empty input")

    digits = [int(digit) for digit in text.split()]

    if any(digit < 0 or digit > 9 for digit in digits):
        raise ValueError("Invalid digit")

    if len(digits) > 1 and digits[0] == 0:
        raise ValueError("Leading zero")

    return digits

def plus_one(digits):
    """Увеличивает число, представленное списком цифр, на единицу."""
    if not digits:
        raise ValueError("Empty list")

    result = digits[:]
    index = len(result) - 1

```

```

while index >= 0:
    if result[index] < 9:
        result[index] += 1
        return result

    result[index] = 0
    index -= 1

return [1] + result

```

test_lab1_functions.py:

```
# pylint: disable=import-error
```

```
"""Тесты к функциям из лабораторной работы 1."""
```

```
import pytest
```

```
from lab1_functions import calculate_median, parse_digits, parse_numbers, plus_one
```

```
def test_parse_numbers():
    """Проверяет преобразование строки в список чисел."""
    assert parse_numbers("1 2 3") == [1, 2, 3]
```

```
def test_parse_numbers_empty():
    """Проверяет ошибку при пустом вводе."""
    with pytest.raises(ValueError):
        parse_numbers("")
```

```
def test_parse_numbers_invalid():
    """Проверяет ошибку при некорректном числе."""
    with pytest.raises(ValueError):
        parse_numbers("1 a 3")
```

```
@pytest.mark.parametrize(
    ("numbers", "expected"),
    [
        ([1, 2, 3], 2),
        ([1, 2, 3, 4], 2.5),
        ([5], 5),
        ([-3, -1, -2], -2),
    ],
)
```

```
def test_calculate_median(numbers, expected):
    """Проверяет вычисление медианы."""
    assert calculate_median(numbers) == expected
```

```
def test_calculate_median_empty():
    """Проверяет ошибку при пустом списке."""
    with pytest.raises(ValueError):
        calculate_median([])
```

```
def test_parse_digits():
    """Проверяет преобразование строки в список цифр."""
    assert parse_digits("1 2 3") == [1, 2, 3]
```

```
@pytest.mark.parametrize("text", ["", "1 10 3", "1 -1 3", "0 1 2"])
```

```
def test_parse_digits_invalid(text):
    """Проверяет ошибки при неверном вводе цифр."""
    with pytest.raises(ValueError):
        parse_digits(text)
```

```
@pytest.mark.parametrize(
    ("digits", "expected"),
    [
        ([1, 2, 3], [1, 2, 4]),
        ([1, 2, 9], [1, 3, 0]),
        ([9], [1, 0]),
        ([9, 9, 9], [1, 0, 0, 0]),
    ],
)
```

```
def test_plus_one(digits, expected):
    """Проверяет увеличение числа на единицу."""
    assert plus_one(digits) == expected
```

```
def test_plus_one_empty():
    """Проверяет ошибку при пустом списке."""
    with pytest.raises(ValueError):
        plus_one([])
```

```

C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6>python -m pytest src/proj2
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6
plugins: anyio-4.13.0
collected 18 items

src\proj2\test_lab1_functions.py ..... [100%]

===== 18 passed in 0.04s =====

```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Напишите метод `String substringBetween(String str, String open, String close)` выделяющий подстроку относительно открывающей и закрывающей строки.

Спецификация метода:

`substringBetween (None , None, None) = TypeError`

`substringBetween (None, * , *) = None`

`substringBetween (* , None, *) = None`

`substringBetween (* , * , None) = None`

`substringBetween ("", "", "") = ""`

`substringBetween ("", "", "]") = None`

`substringBetween ("", "[", "]") = None`

`substringBetween (" yabcz ", "", "") = ""`

`substringBetween (" yabcz ", "y", "z") = " abc"`

`substringBetween (" yabczyabcz ", "y", "z") = " abc "`

`substringBetween ("wx[b]yz", "[", "]") = "b"`

string_utils.py:

```
# pylint: disable=invalid-name
```

```
"""Функция substringBetween для варианта 7."""
```

```
def substringBetween(text, open_token, close_token):
```

```
    """Возвращает подстроку между открывающей и закрывающей строками."""
```

```
    if text is None and open_token is None and close_token is None:
```

```
        raise TypeError("All arguments cannot be None")
```

```
    if text is None or open_token is None or close_token is None:
```

```
        return None
```

```

if open_token == "" and close_token == "":
    return ""

start = text.find(open_token)

if start == -1:
    return None

start += len(open_token)
end = text.find(close_token, start)

if end == -1:
    return None

return text[start:end]

```

test_string_utils.py:

```

# pylint: disable=import-error

"""Тесты для substringBetween, вариант 7."""

import pytest

from string_utils import substringBetween

def test_substring_between_none_none_none():
    """Проверяет TypeError для трех None."""
    with pytest.raises(TypeError):
        substringBetween(None, None, None)

@pytest.mark.parametrize(
    ("text", "open_token", "close_token", "expected"),
    [
        (None, "[", "]", None),
        ("abc", None, "]", None),
        ("abc", "[", None, None),
        ("", "", "", ""),
        ("", "", "]", None),
        ("", "[", "]", None),
        ("yabcz", "", "", ""),
        ("yabcz", "y", "z", "abc"),
        ("yabczyabcz", "y", "z", "abc"),
        ("wx[b]yz", "[", "]", "b"),
    ],
)

def test_substring_between(text, open_token, close_token, expected):
    """Проверяет функцию по спецификации."""
    assert substringBetween(text, open_token, close_token) == expected

```

```

C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6>python -m pytest src/proj3
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\user\Documents\GitHub\spp_po13\reports\Shevchuk\6
plugins: anyio-4.13.0
collected 11 items

src\proj3\test_string_utils.py ..... [100%]

===== 11 passed in 0.04s =====

```

Вывод работы: освоил приемы тестирования кода на примере использования пакета pytest